

immobiliare-photo-lab

Nota di accompagnamento al case study — Pio Santosuosso, settembre 2026

Questa è la nota di accompagnamento del prototipo che ho costruito per il case study *Product Builder, Agentic AI Products*. Il prodotto è volutamente piccolo: la foto di copertina di un annuncio, corretta con un click, senza mai cambiare quello che c'è nella stanza. L'ho costruito per rispondere ai quattro punti del brief nel modo in cui li affronterei su un prodotto vero: definire il problema, strutturare alternative confrontabili, scegliere i parametri di confronto e rendere misurabile ciò che sembra soggettivo. Quello che trovate qui è una simulazione completa di quel metodo, in scala ridotta.

1. Come affronto il problema della qualità immagini

Il primo passo è stato restringere il perimetro. Non ho provato a migliorare tutte le foto di tutti gli annunci: ho scelto la foto di copertina, caricata dal telefono, da un agente immobiliare subito dopo la visita. È l'utente più frequente e meno attrezzato, perché non è un fotografo, non ha tempo e non ha strumenti di fotoritocco, e la copertina è la foto che decide se un annuncio viene aperto oppure no.

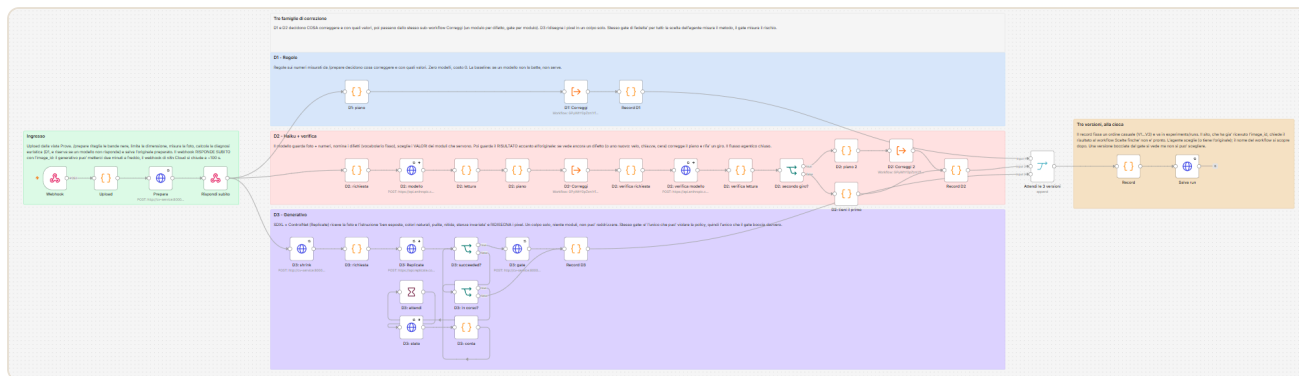
Il bisogno che voglio risolvere è concreto: pubblicare subito senza che la copertina sembri fatta di fretta. Oggi l'alternativa per l'agente è pubblicare la foto così com'è oppure tornare in casa a rifarla.

Il prodotto riceve la foto, dice cosa non va e corregge soltanto quello. I difetti che riconosce sono quelli tipici di uno scatto veloce col telefono: la foto è buia, è in controluce, ha i colori falsati da una lampada, è rumorosa, è storta, è compressa da un'app di messaggistica o è troppo piccola. A ogni difetto corrisponde un modulo di correzione (Colore, Luce, Pulizia, Raddrizza, Nitidezza, Risoluzione) e i moduli partono solo se servono: una foto che è soltanto storta viene soltanto raddrizzata. Alla fine il prodotto spiega all'agente cosa ha fatto e cosa ha lasciato stare, e quando non c'è nulla da correggere lo dice e restituisce l'originale.

Il prodotto non aggiunge, non toglie e non ridisegna nulla. Non cambia il cielo, non rimuove oggetti, non allarga le stanze, non cancella crepe o muffa. Non corregge il mosso, la prospettiva a imbuto del grandangolo o il disordine della stanza, perché non esiste un modo onesto di farlo: li segnala all'agente come consigli. Non tenta di recuperare foto irrecuperabili, ma consiglia di rifarle. L'unica ricostruzione generativa che ammette è la super-risoluzione per le foto sotto i 1000 pixel, e in quel caso l'output viene etichettato come ricostruito e l'agente lo vede.

I rischi che voglio evitare sono cinque: una foto che mente al compratore; un difetto strutturale dell'immobile che sparisce; una correzione che peggiora la foto amplificando rumore, blocchi di compressione o chiazze sui muri; un ritaglio fatto senza dirlo; un agente che non capisce cosa è cambiato. Da questi rischi discendono due scelte di progetto. La prima è che lo spazio delle correzioni è un tipo chiuso: ciò che non ha un parametro non si può chiedere, nemmeno da un modello. La seconda è che ogni correzione passa da un controllo di fedeltà deterministico e bloccante contro l'originale, così che non sia l'intelligenza artificiale a valutare se stessa.

2. Come strutturo il workflow e i flussi da testare

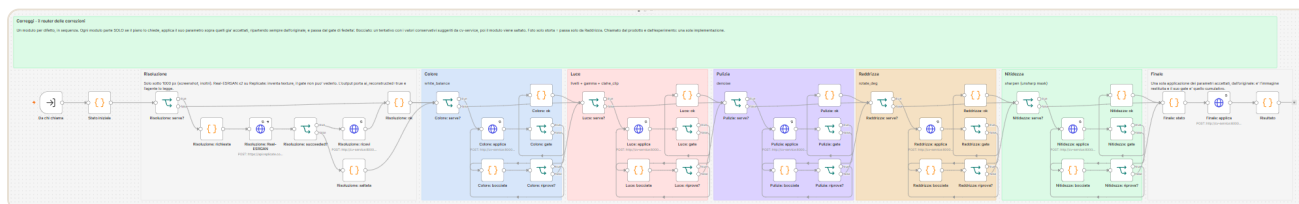


Il workflow del prodotto in n8n: ingresso, tre corsie (D1 regole, D2 Haiku con verifica, D3 generativo), record e salvataggio.

Il workflow ha una forma sola: la foto viene misurata, qualcuno la diagnostica e produce un piano, il piano passa dal modulo Correggi, e ne esce una versione corretta insieme a un registro di quello che è successo. Correggi è un sub-workflow n8n con un modulo per difetto, eseguiti in sequenza; ogni modulo parte solo se il piano lo chiede, ha il proprio controllo di fedeltà e, se viene bocciato, fa un tentativo più prudente e poi rinuncia. Questa parte è identica per tutte le alternative, e questo è il punto: quello che metto a confronto è chi decide cosa correggere, non come si corregge.

Le alternative che ho testato sono tre famiglie diverse, non tre versioni dello stesso modello. La prima, che chiamo D1, decide con regole scritte a mano sui numeri misurati: luminosità media, dominante di colore, rumore previsto dopo lo schiarimento, linee inclinate, livello di compressione. Non chiama nessun modello e costa praticamente zero. La seconda, D2, affida la diagnosi a Claude Haiku 4.5, che guarda la foto insieme ai numeri, nomina i difetti in un vocabolario fisso e sceglie i valori dei moduli; poi guarda il risultato accanto all'originale e, se vede ancora un difetto o ne vede uno nuovo, corregge il piano e rifà un giro. È il flusso agentic chiuso. La terza, D3, è un modello generativo (SDXL con ControlNet, su Replicate) che riceve la foto e un'istruzione in linguaggio naturale e ridisegna i pixel in un colpo solo: non passa da Correggi, ma passa dallo stesso controllo di fedeltà.

Gli output vengono generati dentro lo stesso workflow n8n: ogni foto attraversa le tre famiglie e ogni versione porta con sé il proprio registro, cioè i difetti trovati, il piano, i moduli eseguiti o saltati, la fedeltà, il costo, il tempo e le eventuali correzioni che il sistema ha imposto al piano del modello.

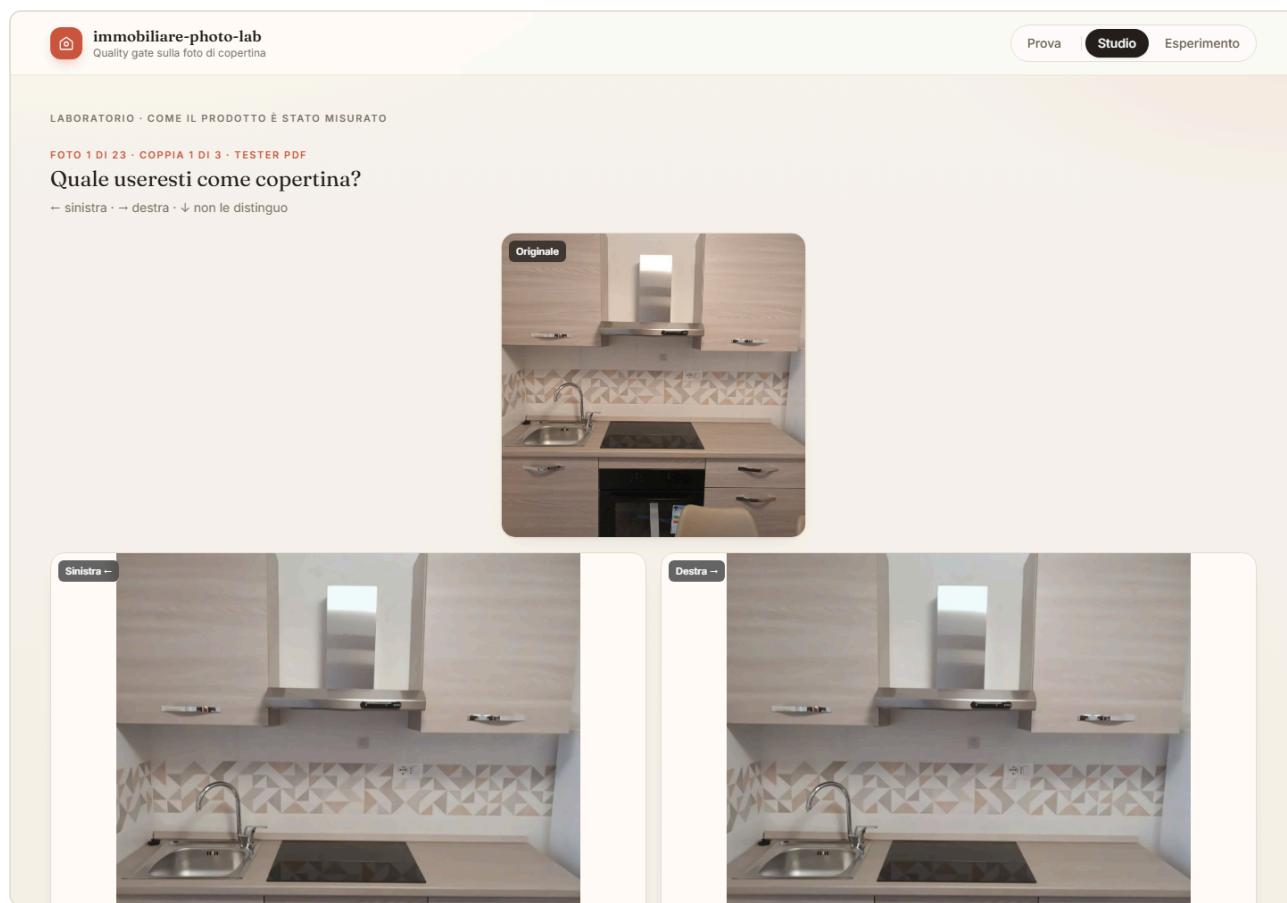


Il sub-workflow Correggi: sei corsie, una per modulo, con controllo di fedeltà e tentativo prudente per ciascuna.

I risultati vengono raccolti dal prodotto stesso. Nella vista Prova le tre versioni arrivano in ordine casuale e senza nome; l'agente sceglie quella che userebbe, oppure tiene l'originale, e solo dopo scopre quale metodo ha prodotto cosa. La vista Studio ripete lo stesso esercizio in forma di test cieco a coppie su un insieme fisso di ventiquattro foto etichettate a mano, per raccogliere giudizi in modo sistematico. Ogni esecuzione e ogni giudizio diventano un record, e la vista Esperimento li aggrega in un tabellone

ricalcolato a ogni apertura. Un quarto workflow, il batch, è la versione pensata per scalare: lavora sul dataset etichettato e può sostituire il valutatore umano con un giudice automatico.

Il metodo migliore non viene scelto guardando le foto: viene scelto con una regola scritta prima di vedere i dati, che descrivo nel punto seguente.



La vista Studio: l'originale in alto, due versioni sotto, giudizio con le frecce.

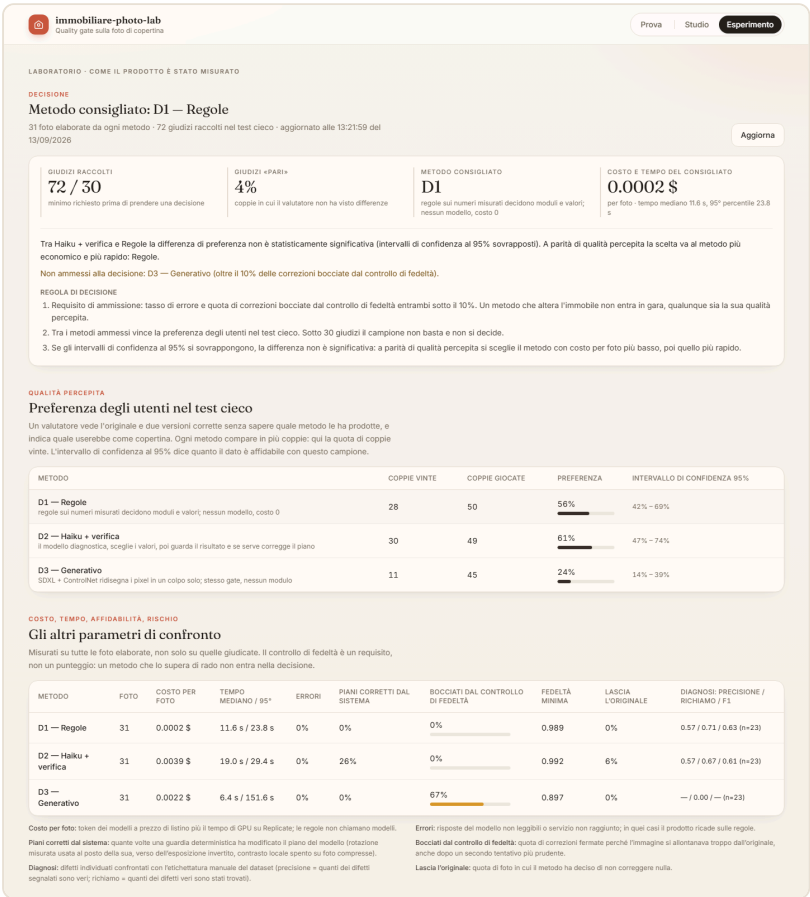
3. Quali parametri uso per valutare che un metodo è migliore dell'altro

Ho scelto sei parametri, ognuno per una ragione precisa. La qualità dell'output è ciò che l'agente vede, e la misuro come preferenza in un test cieco a coppie, con un intervallo di confidenza. Il costo per immagine decide se il prodotto può scalare a migliaia di annunci al giorno, e lo misuro sommando i token dei modelli a prezzo di listino e il tempo di GPU. Il tempo di elaborazione conta perché l'agente sta caricando l'annuncio in quel momento, e lo riporto come mediana e novantacinquesimo percentile. L'affidabilità del workflow la misuro contando le risposte illeggibili dei modelli, le volte in cui il prodotto è dovuto ricadere sulle regole e le volte in cui una guardia deterministica ha dovuto correggere il piano del modello. Il rischio di alterare l'immobile non è un punteggio ma un vincolo: conto le correzioni bocciate dal controllo di fedeltà, la fedeltà minima osservata e la percentuale di ritaglio dichiarata. Infine misuro l'accuratezza della diagnosi, cioè precisione e richiamo dei difetti individuati rispetto alle etichette manuali, perché separa "ha capito il difetto" da "l'ha corretto bene".

La regola di decisione è in tre passi. Primo, non è ammesso alla decisione un metodo che supera il dieci per cento di errori o il dieci per cento di correzioni bocciate dal controllo di fedeltà: il rischio non si

scambia con la qualità. Secondo, tra i metodi ammessi vince la preferenza nel test cieco, purché ci siano almeno trenta giudizi. Terzo, se gli intervalli di confidenza si sovrappongono la differenza non è significativa, e a parità di qualità percepita si sceglie il metodo più economico, poi quello più rapido.

Nella simulazione, con ventiquattro foto, sessantotto giudizi e un solo valutatore, leggerei i dati così. Il generativo esce al primo passo, perché il controllo di fedeltà boccia sei versioni su dieci e perché quasi non corregge l'esposizione. Regole e modello risultano indistinguibili sulla preferenza, sessantuno contro cinquantotto per cento con intervalli sovrapposti e undici a dieci nello scontro diretto, quindi il terzo passo indicherebbe le regole. Con un solo valutatore, però, la conclusione corretta è che servono altri giudizi, non che abbiamo scelto: ed è esattamente quello che il tabellone dice.



La vista Esperimento: decisione, regola applicata, preferenza nel test cieco e gli altri parametri.

4. Come misuro gli elementi qualitativi

"Foto migliore" non ha un numero, ma ha una preferenza che si può misurare se la si raccoglie nel modo giusto. Il valutatore vede l'originale in alto e due versioni corrette sotto, i lati sono assegnati a caso, non compare nessun nome e nessun dettaglio prima della scelta, ed è ammesso rispondere che le due versioni sono indistinguibili. Ogni metodo compare in più coppie per ogni foto, e sulla quota di coppie vinte calcolo un intervallo di confidenza di Wilson, per non leggere differenze che il campione non regge. Più valutatori significano più foto giudicate, non più opinioni sulla stessa foto.

Il realismo non lo misuro con un voto, perché è un requisito e non una qualità da graduare. Lo verifica il controllo di fedeltà, che confronta la struttura dell'immagine corretta con quella dell'originale blocco per blocco e controlla che i colori non siano stati sostituiti. L'ho calibrato su cinquanta correzioni lecite e

cento vietate costruite sulle stesse foto: le lecite passano tutte, le vietate passano in sette casi su cento, e sono tutte crepe sottili, un limite che dichiaro. Per scalare oltre i valutatori umani, il workflow batch prevede un giudice automatico a coppie, con tre ripetizioni e voti non unanimi contati come pareggio; ma un giudice automatico va validato contro i giudizi umani prima di sostituirli, e questo passo nella simulazione non l'ho ancora fatto.



La vista Prova, il prodotto: si carica una foto e si scelgono le versioni alla cieca.

Prototipo, strumenti, cosa è reale e cosa no

Ho costruito il prototipo con n8n Cloud per l'orchestrazione, un servizio locale in Python (FastAPI e OpenCV) per la pipeline di correzione, il controllo di fedeltà e i dati, e un sito in React per l'agente e per il laboratorio. I modelli sono Claude Haiku 4.5 per la diagnosi e la verifica, SDXL con ControlNet e Real-ESRGAN su Replicate per la parte generativa, e Claude Sonnet 5 come giudice del batch. Ho lavorato con Claude Code.

Sono reali tutte le chiamate ai modelli, le ventiquattro foto (case vere prese da annunci pubblici), il controllo di fedeltà e la sua calibrazione, i sessantotto giudizi, i costi e i tempi misurati. Sono simulati o limitati il numero di valutatori, che è uno ed è l'autore; i costi, calcolati a listino; le latenze, misurate attraverso un tunnel gratuito; e il giudice automatico, che è pronto ma non è stato eseguito sull'intero insieme.

I limiti principali sono questi. Il controllo di fedeltà non vede le strutture sotto i tre pixel e non vede la super-risoluzione, che per questo viene etichettata. Il rilevatore di inclinazione rinuncia quando le linee stanno tutte da un lato della foto, perché in quel caso non distingue una rotazione dalla prospettiva. n8n Cloud chiude i webhook dopo circa cento secondi, quindi il prodotto risponde subito e il sito va a chiedere il risultato. Il campione di foto è piccolo e poco difettoso, perché sono foto già pubblicate.

I prossimi passi che farei sono, nell'ordine: raccogliere giudizi da più valutatori sullo stesso insieme prima di qualunque decisione tra regole e modello; aggiungere un modulo Privacy per sfocare volti e targhe in modo deterministico; validare il giudice automatico sui giudizi umani; ricalibrare le soglie del controllo di fedeltà su un campione più ampio; e poi fare il test che conta davvero, cioè la copertina corretta contro l'originale, misurata sui contatti ricevuti.

Appendice tecnica

Com'è fatto

Il sistema ha tre parti. Il sito React (viste Prova, Studio ed Esperimento) parla soltanto con i webhook di n8n, quindi nessuna chiave arriva nel browser. n8n Cloud orchestra e decide: riceve l'upload, chiama i modelli, esegue il sub-workflow Correggi e salva i record; tra i nodi viaggiano identificativi, numeri e, nel prodotto, la foto in base64. Il servizio cv-service, in Python 3.12 con FastAPI e OpenCV, gira in locale ed è raggiunto da n8n attraverso un tunnel ngrok; fa tutto ciò che è deterministico, cioè la pipeline di correzione (`app/core/pipeline.py`), il controllo di fedeltà (`app/core/metrics.py`), il dataset e i record (`app/api/dataset.py`), i giudizi e il tabellone (`app/api/experiment.py`). Ha trentacinque test che si eseguono con `pytest`.

I workflow n8n non sono stati disegnati a mano ma generati da codice: `n8n/build_workflows.py` produce i quattro file JSON leggendo i prompt da `prompts/*.md` e gli indirizzi da `.env`. Se cambia un prompt o l'URL del tunnel, si rigenera e si reimporta. Il contratto dati è unico ed è descritto in `docs/contracts.md`; le sue due implementazioni sono `cv-service/app/schemas.py` e `frontend/src/types.ts`.

Dove gira

Il prototipo è pubblicato e funziona senza nulla di acceso sul computer di chi lo ha costruito. Il sito è su Vercel (<https://immobiliare-photo-lab.vercel.app>) e si ricostruisce da solo a ogni push. n8n è self-hosted su un VPS Hostinger, in Docker, raggiungibile in https; cv-service gira sullo stesso server come container sulla rete interna di n8n, con `dataset/` ed `experiments/` montati dal repository clonato sul server, e si aggiorna con uno script (`deploy/update.sh`). Il codice è su GitHub: <https://github.com/Pio57/immobiliare-photo-lab>.

Come si avvia in locale

Servono Python 3.12, Node 20 o superiore, un account n8n Cloud (oppure n8n in locale con il `docker-compose.yml` incluso) e ngrok.

Prima si copia `.env.example` in `.env` e si compilano i valori descritti nella sezione seguente. Poi si avvia cv-service:

```
cd cv-service
py -3.12 -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pytest
uvicorn app.main:app --port 8000
```

In una seconda finestra si apre il tunnel con `ngrok http 8000` e si copia l'indirizzo pubblico in `.env` come `CV_SERVICE_PUBLIC_URL`. A quel punto si generano i workflow con `cv-`

`service\.venv\Scripts\python.exe n8n\build_workflows.py` e si importano in n8n nell'ordine descritto in `n8n/README.md`: prima Correggi, di cui si copia l'identificativo in `.env` come `N8N_CORRECT_WORKFLOW_ID`, poi, dopo aver rigenerato, il prodotto, le scelte e il batch. A ogni workflow vanno assegnate le credenziali e va fatto Publish. Infine il sito: `cd frontend`, `npm install`, `npm run dev`, e si apre `http://localhost:5173`. Ogni volta che ngrok viene riavviato l'indirizzo cambia, e bisogna aggiornare `.env`, rigenerare e reimportare.

Quali chiavi servono

`ANTHROPIC_API_KEY` serve per la diagnosi e il secondo sguardo di Claude Haiku 4.5 e per il giudice del batch, Claude Sonnet 5; in n8n diventa la credenziale "Anthropic" da selezionare sui nodi `D2: modello`, `D2: verifica modello` e `Judge: Sonnet`. `REPLICATE_API_TOKEN` serve per il generativo D3 e per il modulo Risoluzione; in n8n diventa una credenziale "Header Auth" con `Authorization: Bearer <token>` sui nodi `D3: Replicate`, `D3: stato` e `Risoluzione: Real-ESRGAN`. `CV_SERVICE_PUBLIC_URL` è l'indirizzo ngrok di cv-service e viene compilato dentro i workflow. `N8N_CORRECT_WORKFLOW_ID` è l'identificativo del sub-workflow Correggi. `VITE_N8N_WEBHOOK_URL` è il webhook del prodotto, dal quale il sito ricava gli altri tre.

Le chiavi non entrano mai nel repository, perché `.env` è ignorato da git, e non arrivano mai al browser, perché le chiamate ai modelli partono da n8n. I costi osservati sono di circa 0,004 dollari a foto per D2, 0,002 per D3 e 0,003 per ogni coppia giudicata dal giudice automatico; l'intero lavoro è costato circa dieci dollari di API.

I workflow n8n nel dettaglio

Correggi (`workflow-correct.json`, sub-workflow, cinquanta nodi) è il router delle correzioni. Riceve la foto, il piano e il nome con cui salvare il risultato. Il nodo `Stato iniziale` prepara il registro dei sei moduli. Seguono sei corsie in sequenza, tutte con lo stesso schema: il nodo `serve?` controlla se il piano chiede quel parametro; `applica` chiama `/apply` con i parametri già accettati più quello in esame, ripartendo sempre dall'originale; `gate` verifica che la fedeltà globale sia almeno 0,90 e che il blocco peggiore sia almeno 0,65; se passa, `ok` aggiunge il parametro agli accettati, altrimenti `bocciata` prepara un tentativo con i valori conservativi suggeriti da cv-service e `riprova?` rimanda ad `applica` una sola volta, dopodiché il modulo viene saltato. Questo ciclo è visibile sul canvas. La prima corsia, Risoluzione, è l'unica generativa: usa Real-ESRGAN a fattore due, solo per foto sotto i 1000 pixel, e quando interviene l'immagine ingrandita diventa la foto di lavoro dei moduli successivi, con il piano adattato (niente denoise, contrasto locale al massimo 1, nitidezza al massimo 0,2) e l'etichetta `ai_reconstructed`. Il nodo `Finale` applica una volta sola tutti i parametri accettati e `Risultato` restituisce la foto, i parametri, il registro dei passi, la percentuale di ritaglio e l'etichetta.

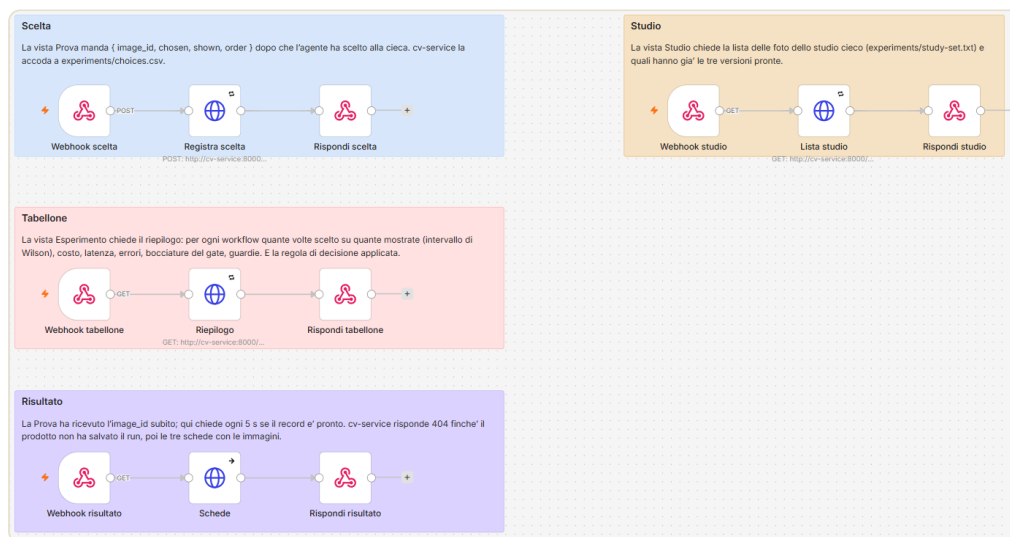
Prodotto (`workflow-product.json`, webhook `photo-lab-product`, trentanove nodi) è quello che il sito chiama a ogni upload. `Upload` legge il file, `Prepara` chiama `/prepare`, che rimuove le bande nere, limita la dimensione, misura la foto, produce gli avvisi, calcola la diagnosi euristica e salva l'originale preparato. Subito dopo `Rispondi subito` restituisce al sito l'identificativo dell'esecuzione, perché n8n Cloud chiude i webhook dopo circa cento secondi e il generativo a freddo può impiegarne centoventi; il sito chiede poi il risultato ogni cinque secondi. Da qui partono le tre corsie.

Nella corsia D1 il nodo **D1: piano** prende i parametri euristici già calcolati da **/prepare** (la luminanza media determina la gamma, la dominante il bilanciamento del bianco, il rumore previsto dopo lo schiarimento il denoise, le linee concordi la rotazione, la compressione spegne il contrasto locale e forza il deblock) e li manda a **D1: Correggi**; **Record D1** scrive il record.

Nella corsia D2 il nodo **D2: richiesta** costruisce la chiamata a Claude Haiku 4.5 con la foto, i numeri e il prompt di **prompts/diagnosis.md**; **D2: lettura** interpreta il JSON di risposta, che contiene i difetti nel vocabolario fisso, l'esposizione espressa in stop, i valori dei moduli, i consigli non correggibili, la raccomandazione (applicare, applicare con prudenza, lasciare l'originale) e un motivo in italiano per l'agente. **D2: piano** applica le guardie deterministiche: se il rilevatore ha misurato un'inclinazione, vince la misura; se il modello chiede di scurire una foto scura, il verso viene invertito; su una foto compressa il contrasto locale viene spento e la nitidezza limitata. Ogni intervento finisce in **plan_fixes** e viene contato nel tabellone. Dopo **D2: Correggi**, il nodo **D2: verifica richiesta** mostra al modello l'originale, il risultato e il registro; se il modello risponde **adjust**, **D2: piano 2** e **D2: Correggi 2** rifanno la correzione con il piano nuovo, altrimenti **tieni il primo** conserva la prima. **Record D2** somma costi e tempi dei due giri.

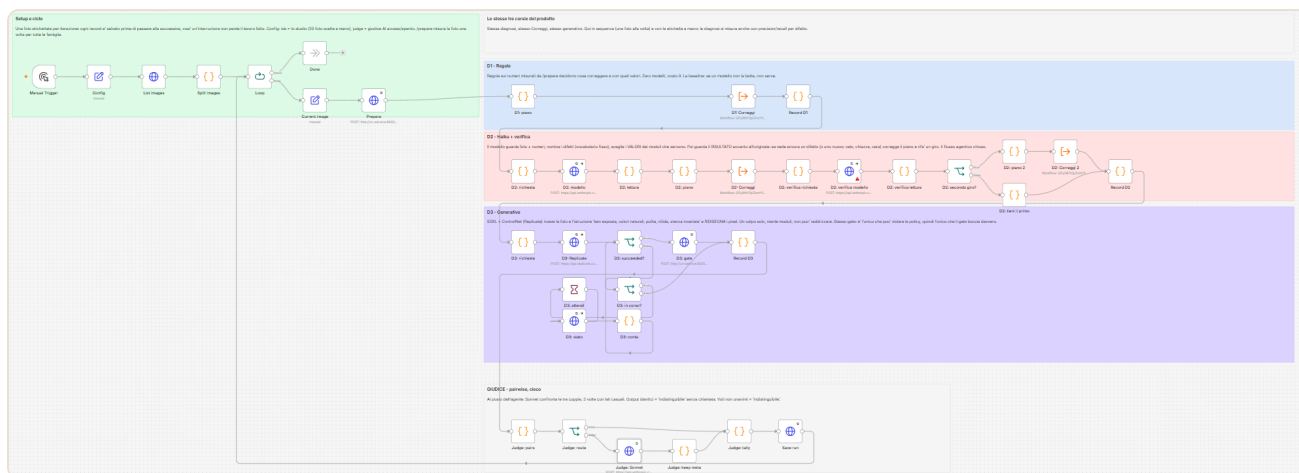
Nella corsia D3 il nodo **D3: shrink** porta la foto a 1024 pixel per rispettare il limite del data URI di Replicate, **D3: richiesta** compone la chiamata a SDXL con ControlNet con un prompt fisso che chiede la stessa stanza ben esposta, con colori naturali, pulita e nitida, e **D3: Replicate** la invia. Se la predizione è ancora in avvio quando la risposta torna, il ciclo **in corso?**, **attendi**, **stato**, **conta** la interroga ogni quindici secondi fino a otto volte. **D3: gate** chiama **/gate_remote**, che scarica l'immagine, la confronta con l'originale, la salva e misura a posteriori cosa è cambiato, perché un modello generativo non dichiara le proprie scelte. **Record D3** scrive il record con le sei righe misurate.

Il nodo **Attendi le 3 versioni** aspetta le tre corsie, **Record** fissa un ordine casuale delle versioni e **Salva run** scrive tutto in **experiments/runs**.



I workflow Scelte (quattro webhook) e Batch (dataset con giudice automatico opzionale).

Scelte (**workflow-choice.json**, dodici nodi) espone quattro webhook: **photo-lab-choice** registra un giudizio in **experiments/choices.csv**; **photo-lab-result** restituisce le tre versioni di un'esecuzione e risponde che non è pronta finché il record non esiste; **photo-lab-study** restituisce l'elenco delle foto dello studio e il loro stato; **photo-lab-summary** restituisce il tabellone con la regola di decisione già applicata.



Il workflow Batch: le stesse tre corsie su tutto il dataset, una foto alla volta.

Batch (`workflow-batch.json` , quarantacinque nodi, avvio manuale) ripete le tre corsie del prodotto sul dataset, una foto alla volta, con un nodo `Config` che elenca le foto dello studio, permette di riprendere un'esecuzione interrotta e accende o spegne il giudice automatico, cioè Claude Sonnet 5 che confronta le coppie tre volte con lati casuali e considera pareggio i voti non unanimi. Le ventiquattro foto dello studio sono state generate così, con il giudice spento per usare i giudizi umani.

I dati prodotti

Ogni foto elaborata lascia un record in `experiments/runs/<id>.json` con le tre versioni, i difetti, il piano, il registro dei moduli, la fedeltà, il costo, il tempo e le correzioni imposte dal sistema, oltre all'ordine cieco con cui le versioni sono state mostrate. Le immagini stanno in `dataset/processed/` , una per versione più l'originale preparato. Ogni giudizio è una riga di `experiments/choices.csv` con la foto, la coppia mostrata, la scelta o il pareggio e le iniziali del valutatore. La calibrazione del controllo di fedeltà si rigenera con `experiments/scripts/calibrate_gate.py` e il suo output è in `experiments/gate-calibration.txt` . Il tabellone della vista Esperimento è la risposta di `GET /summary` , calcolata da questi file.

Struttura del repository

cv-service/	FastAPI e OpenCV: pipeline, controllo di fedeltà, dataset, record, giudizi, tabelle
n8n/	workflow-correct, workflow-product, workflow-choice, workflow-batch, build_workflow
frontend/	React: Prova (prodotto), Studio (test cieco), Esperimento (tabellone)
prompts/	diagnosis.md (diagnosi e secondo sguardo), judge.md (giudice automatico del batch)
dataset/	le 24 foto e labels.csv; experiments/ contiene record, giudizi e calibrazione
docs/	contracts.md (contratto dati) e gate-calibration.md